IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of submission October 27, 2026.

Digital Object Identifier to be assigned by IEEE

Optimize Before You Compress: SSA-Based Compaction of Three-Address Code, Verified by Differential Testing

HRITURAJ DEB[1], GAUTHAM G[1], RISHABH KUMAR[1], and AARNAV BHASIN[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, India (e-mail: hrituraj.deb2024@vitstudent.ac.in; gautham.g2024@vitstudent.ac.in; rishabh.kumar2024@vitstudent.ac.in; aarnav.bhasin2024@vitstudent.ac.in)

Corresponding author: Hrituraj Deb (e-mail: hrituraj.deb2024@vitstudent.ac.in).

This work received no external funding.

ABSTRACT Software increasingly reaches phones, browsers and embedded devices as portable intermediate code, often as small plug-ins and over-the-air updates, where every byte counts. Three-address code is verbose: literals sit in fresh temporaries, values are recomputed, and dead branches survive. Existing work either shrinks native code inside large production compilers or compresses intermediate code as an opaque file with Brotli, Zstandard or xz, and reported savings rarely separate optimization from encoding. We present a pipeline that converts three-address code to static single assignment form, runs seven passes, including sparse conditional constant propagation and copy propagation, to a fixed point, and writes a self-describing bytecode with variable-length operands. We evaluate seven demonstration programs and ten C kernels that read input and call functions, checking every result with independent decoders, reference interpreters and differential testing. Optimization removed 48.1% to 89.3% of the instructions of the demonstration programs, but only 19.0% to 34.3% of the kernels. Against the best of Brotli, Zstandard and xz applied to the same optimized program as text, the bytecode was 3.4% to 59.7% smaller on the demonstration programs and 39.5% to 50.9% smaller on the kernels; with a shared dictionary, the compressors won on 5 of the 7 small programs. Canonical Huffman coding paid off above about 115 bytes. Differential testing showed that the first release miscompiled 213 of 1,000 random programs (21.3%) by folding division with floor semantics; the corrected pipeline agrees on all of them. Future work will add arrays and pre-trained entropy models.

INDEX TERMS Bytecode compression, compiler optimization, differential testing, Huffman coding, intermediate representation, static single assignment.

I. INTRODUCTION

PORTABLE intermediate code has become a common way to deliver software. Embedded and Internet of Things (IoT) devices receive new functionality through over-the-air updates, where every transmitted byte costs radio energy and every stored byte costs flash memory [1]. Web browsers execute WebAssembly modules, a standardized portable bytecode [2], whose performance and deployment have been studied on large application suites [3]. Compiler infrastructures such as MLIR keep programs in several levels of intermediate representation (IR) before any machine code is produced [4]. Most of these representations build on static single assignment (SSA) form, in which every variable is defined exactly once [5].

The problem studied in this article is the size of such intermediate code when it must be stored or sent. A straightforward lowering of a C-like program to three-address code (TAC) is verbose. Every literal is placed in a fresh temporary, expressions whose operands are compile-time constants are left for run time, and branches that can never be taken are kept. Our motivation is practical. A teaching compiler built by our team, called IR (intermediate representation) Compression Studio, reported large end-to-end savings, but it was unclear how much of each saving came from optimization, how much from the binary format, and whether every reported number and every optimization was correct. Small programs make the question sharper, because fixed per-file costs such as coder tables and container headers can outweigh the payload. Programs whose values are all known at compile time make it misleading, because optimization then removes almost everything.

Existing methodologies attack program size from two di-

rections. The first direction transforms the program. Classic scalar optimizations such as constant propagation, value numbering and dead-code elimination are textbook material [6], and SSA form makes them sparse and fast [5]. Production compilers add size-specific transformations and increasingly choose among them with machine learning [7], [8].

The second direction encodes the program compactly after it has been produced. Entropy coders such as Huffman codes assign short bit strings to frequent symbols and are still widely used inside larger compressors [9]. General-purpose compressors combine such coders with string matching and can be applied to any IR file without knowing what it contains.

Both directions have shortfalls for small intermediate programs. Size-oriented compiler transformations, such as function merging, loop rolling, inlining and outlining [10]–[15], are built into large production toolchains and are evaluated on native binaries of whole applications, so their gains are hard to reproduce in a small educational compiler or to separate from the binary format. Machine-learned approaches [16]–[20] add training infrastructure and models that are larger than the programs they optimize.

General-purpose compressors are blind to program semantics. They cannot remove an instruction that is never executed, and on inputs of a few hundred bytes their containers and coding tables are a large fraction of the output. Neither direction routinely checks that the transformed program still behaves like the original, although compilers are known to contain miscompilation bugs that only systematic testing reveals [21].

This article proposes an SSA-based compaction pipeline that optimizes first and encodes second. The program, which may define functions and read input, is lowered to TAC, converted to semi-pruned SSA form and optimized with seven passes iterated to a fixed point: constant folding and propagation, sparse conditional constant propagation (SCCP) [22], copy propagation, dominator-scoped value numbering, dead-code elimination, CFG simplification and trivial-ϕ removal. The optimized code is then written as a compact, self-describing bytecode, which we call CLIR, using one opcode byte and Little Endian Base 128 (LEB128) operands [2]. Its constant pool holds exactly the literals the code refers to, and it can be followed by a canonical Huffman coder [23] or replaced by a column-oriented container. The approach is apt because the programs are small and many of their values are known at compile time, which is where removing instructions beats compressing them; programs that read input show how much of that advantage remains.

The main advantage of the proposed work is that its claims are checked rather than assumed. Every number in this article is produced by evaluation scripts released with the compiler. Every encoding is decoded without any table from the encoder and executed again; the behavior of every program is compared by reference interpreters, by native compilation of the kernels and by differential testing. The contributions are: 1) a reproducible SSA compaction pipeline with seven passes and an exact, self-describing byte layout; 2) an evaluation on seven demonstration programs, ten C kernels that read input and call functions, and random programs, which separates the effect of optimization from that of encoding by compressing the same optimized program with Brotli, Zstandard and xz, with and without container headers and shared dictionaries; 3) a comparison with Java class files and LLVM bitcode; 4) a break-even model for static Huffman coding that shows the effect of the header design; and 5) a differential-testing campaign that measured the severity of a constant-folding defect and confirmed its fix.

The rest of the manuscript is structured as follows. Section II reviews fifteen related works and earlier work on code and bytecode compression. Section III formulates the problem and presents the architecture, methodology, modules, flow, equations and algorithms of the proposed system. Section IV describes the implementation tools, parameter setup and performance metrics, and analyzes the results against existing compressors. Section V concludes and outlines future work.

II. RELATED WORKS

This section reviews fifteen recent works in four groups: compiler transformations that reduce code size, machine-learning-guided optimization, studies of portable bytecode, and compiler correctness. For each work we summarize the proposed solution, the methodology and tools, the results, and the research gap that remains for compact intermediate code. Table 1 gives an overview. Section II-E then summarizes the earlier literature on code and bytecode compression and the binary formats of WebAssembly, LLVM and the Java virtual machine, with which our encoding is compared in Section IV.

A. CODE-SIZE REDUCTION BY COMPILER TRANSFORMATIONS

1) SalSSA, function merging in SSA form

Solution. Rocha *et al.* [10] proposed SalSSA, which merges similar functions into one while staying in SSA form, so that the register demotion required by the earlier FMSA approach is no longer needed. The merged control-flow graph is built top-down from the two input graphs, and a ϕ-node coalescing step reduces the extra code that merging introduces.

Methodology and tools. Each function is linearized into labels and instructions, and a sequence alignment pairs matching instructions. ϕ-nodes travel with their block labels, mismatched operands are chosen with a `select` on a function identifier, and the operands of commutative instructions are reordered to avoid selects.

SalSSA was implemented in LLVM 11 and evaluated on SPEC CPU2006 and CPU2017 on an Intel x86 machine, and on MiBench for ARM Thumb, against FMSA. Object size, compile time, peak memory and run time were measured over five repetitions.

Results. Code size fell by 9.3% to 9.7% on SPEC CPU2006 and 7.9% to 9.2% on CPU2017, about twice the reduction of FMSA. Compile-time overhead was about 5% instead of 14%, and memory use roughly halved.

Research gap. The work targets whole functions in native binaries; it does not consider how the remaining IR is encoded.

2) HyFM, hybrid function merging

Solution. HyFM [11] aligns functions block by block instead of as whole instruction sequences. A multi-tier profitability analysis discards unprofitable block pairs before any merged code is generated.

Methodology and tools. Blocks and functions are summarized by vectors of opcode counts, paired by Manhattan distance, and aligned either with the Needleman–Wunsch algorithm or with a linear pairwise alignment of equal-sized blocks. The first tier compares the estimated cost of each block pair before and after merging.

HyFM was implemented in LLVM 11, the same version as SalSSA, and evaluated on SPEC CPU2006 and CPU2017 on an Intel Xeon machine with five repetitions. Code size, merging time, compile time and peak memory were reported.

Results. With pairwise alignment, the code-size reduction stayed within one percentage point of SalSSA, while function merging ran 4.59 to 5.28 times faster on average and peak memory dropped from 2.4 GB to about 610 KB.

Research gap. Block pairing is still quadratic in the number of blocks, and the work remains specific to native code.

3) RoLAG, loop rolling

Solution. RoLAG [12] rolls isomorphic straight-line code back into loops. It finds repeated code by aligning SSA graphs bottom-up from seed instructions such as stores and calls.

Methodology and tools. Mismatching values become loop inputs, and special patterns such as monotonic integer sequences, chained dependences and reduction trees widen the code that can be aligned. A scheduling analysis checks that reordering is legal, and a profitability analysis based on LLVM's code-size cost model keeps the smaller version.

RoLAG was built on LLVM 12 and evaluated with `-Os` on about one million AnghaBench functions, on MiBench and SPEC CPU2017 programs, and on the TSVC kernels, against LLVM's loop-rerolling pass.

Results. Affected AnghaBench functions shrank by 9.12% on average, and the largest absolute saving on full programs was 87.91 KB. On TSVC, RoLAG reduced size by 23.4% against 13.69% for LLVM.

Research gap. Rolling works only inside a single basic block and can be unprofitable when the cost model misses later passes.

4) Inlining for code size

Solution. Damásio *et al.* [13] showed that inlining, usually seen as a code-growing transformation, can shrink code. They inline functions that have a single call site and then delete the original, and they search for calls at which inlining lets the compiler simplify the body for that particular caller.

Methodology and tools. The first technique is a simple heuristic over the call graph. The second is an analysis that estimates whether specializing the callee to the constants at a call site will remove code.

Both techniques were implemented in LLVM and evaluated on MiBench against `clang -Os`, with running time compared statistically.

Results. Code size fell by 2.96% on average and by up to 11%, with no statistically significant change in running time.

Research gap. The evaluation covers one benchmark suite and native code only.

5) Optimal function inlining

Solution. Theodoridis *et al.* [14] studied how far LLVM's inliner is from optimal when optimizing for binary size and derived an autotuning strategy from the findings.

Methodology and tools. A formulation of the inlining problem shrinks the search space enough to evaluate every inlining choice exhaustively for 1135 SPEC2017 files.

The autotuner was then applied to SPEC2017, the LLVM code base and SQLite, with LLVM's heuristic as the baseline.

Results. The authors report an average improvement of 7% (up to 28%) over LLVM on SPEC2017, 15% on LLVM's own code and 10% on SQLite.

Research gap. The study focuses on one transformation in a production compiler and does not address the encoding of portable code.

6) Code-size optimization of production iOS applications

Solution. Chabbi *et al.* [15] reduced the size of a production iOS application written in Swift and Objective-C with whole-program machine-code outlining, repeated over several rounds.

Methodology and tools. An analysis pass logged repeated instruction sequences and traced them to causes such as reference-counting calls, frame setup and copies produced when leaving SSA form.

All modules were compiled to LLVM IR, linked with `llvm-link`, optimized and then outlined as one program, using Swift 5.2, Xcode and patched LLVM for AArch64 devices, with production data from millions of users.

Results. The code section of the UberRider application shrank by 23% (114.5 MB to 88.4 MB) and the whole binary by 17.6%; the Linux kernel and Clang shrank by 14% and 25%. Release builds took 66 instead of 21 minutes.

Research gap. The approach needs a custom toolchain and much longer builds, and it works on machine code rather than on portable IR.

B. MACHINE-LEARNING-GUIDED OPTIMIZATION

1) MLGO

Solution. MLGO [16] is a framework for using machine-learned policies inside LLVM. Its first application replaces the heuristic in LLVM's inlining-for-size pass.

Methodology and tools. Inlining is modeled as a sequential decision process with eleven numerical features of caller, callee and call graph. Policies are trained with policy gradient,

warm-started by imitating the existing heuristic, and with evolution strategies.

Training used TF-Agents on more than 28,000 IR modules of an internal application, and the released model is compiled ahead of time into LLVM, so that compilation stays deterministic.

Results. The learned policies reduced size by up to 7% compared with `-Oz`, for about 1% extra compile time.

Research gap. The method needs large training infrastructure and targets native size only.

2) POSET-RL

Solution. POSET-RL [17] uses reinforcement learning to choose the order of optimization passes so that code size and execution time both improve.

Methodology and tools. A Double Deep Q-Network observes 300-dimensional IR2Vec embeddings of the program and selects sub-sequences of the `-Oz` pipeline. These come from a manual grouping or from an Oz Dependence Graph. The reward combines object-size reduction with throughput estimated by LLVM-MCA.

The agent was trained in LLVM 10 on 130 files from the LLVM test suite and evaluated on MiBench, SPEC CPU2006 and SPEC CPU2017 on x86 and AArch64.

Results. The graph-based variant improved code size by 6.19% and execution time by 11.99% over `-Oz` on SPEC CPU2017 on average.

Research gap. The rewards are proxies, and some benchmarks grew or slowed down.

3) Large language models for compiler optimization

Solution. Cummins *et al.* [18] trained a 7-billion-parameter transformer to read unoptimized LLVM IR and emit a list of passes that minimizes the instruction count.

Methodology and tools. The model follows the Llama 2 architecture and was trained from scratch on one million LLVM-IR functions (about 373 million tokens). Auxiliary tasks ask it to predict instruction counts and the optimized code.

Training labels came from an autotuner that ran tens of thousands of compilations per program. Evaluation used 100,000 unseen functions and compared the model with AutoPhase and Coreset-NVP.

Results. The model's pass lists produced 3.01% fewer instructions than `-Oz` without extra compilations, and its generated code compiled 90.3% of the time.

Research gap. The model cannot reliably fold constants, its context is limited to 2048 tokens, and inference is far slower than compilation.

4) CompilerGym

Solution. CompilerGym [19] packages compiler optimization tasks as reinforcement-learning environments with the OpenAI Gym interface.

Methodology and tools. A Python front end talks to compiler services over remote procedure calls. Environments cover LLVM phase ordering with 124 passes, GCC flag tuning and CUDA loop nests. Observations range from instruction counts to graph representations, and rewards include IR instruction count relative to `-Oz`.

More than one million LLVM benchmarks are bundled, and the framework validates semantics with differential testing and sanitizers.

Results. An environment step took 7.5 ms instead of 205.9 ms in earlier work, about 27 times faster. The best autotuner reached 1.083 times the code-size reduction of `-Oz` on cBench.

Research gap. It is an experimentation platform, and its learning agents generalized poorly across program domains.

5) ProGraML

Solution. ProGraML [20] is a language-independent graph representation of programs for machine learning, combining control, data and call relations in one multigraph.

Methodology and tools. Instructions, identifiers and immediate values become vertices built from LLVM or XLA IR, with typed and ordered edges. A message-passing neural network learns over the graph.

Classic data-flow analyses such as reachability, dominators and liveness were posed as supervised learning tasks on a large dataset of IR files, and the representation was then applied to downstream optimization tasks.

Results. The authors found that earlier learned representations struggle to reproduce data-flow analyses and that ProGraML improves results on downstream tasks.

Research gap. It learns analyses that a compiler can compute exactly, and it does not produce compact program encodings.

C. STUDIES OF PORTABLE BYTECODE

1) Real-world WebAssembly binaries

Solution. Hilbig *et al.* [24] conducted a large empirical study of WebAssembly binaries found in the wild.

Methodology and tools. The authors collected 8461 unique binaries from source repositories, package managers and live websites.

They analyzed source languages, security-relevant imports and use cases, and released the collection, 58 times the size of any WebAssembly benchmark used before.

Results. Two thirds of the binaries were compiled from memory-unsafe languages, 21% imported potentially dangerous host functions, and 29% of the binaries on websites were minified.

Research gap. The study characterizes binaries but does not try to make them smaller.

2) Superoptimization of WebAssembly bytecode

Solution. Cabrera Arteaga *et al.* [25] superoptimized WebAssembly code by inserting the Souper superoptimizer into an LLVM-to-WebAssembly pipeline.

Methodology and tools. C programs were compiled with Clang at `-O3` to LLVM bitcode, passed through Souper,

which proves replacements equivalent with an SMT solver, and linked to WebAssembly.

Four Souper configurations with a 300-second timeout were tried on 12 Rosetta Code programs, and the instruction counts of the resulting modules were compared.

Results. The median reduction was 0.33% of the instructions. One program shrank from 45 to 21 instructions, and one program grew.

Research gap. The cost model works on LLVM IR rather than on the final bytecode, and the evaluation is small.

D. COMPILER CORRECTNESS

1) Alive2

Solution. Alive2 [26] checks automatically, without changes to LLVM, that each optimized function refines the original one, a technique known as translation validation.

Methodology and tools. Source and target functions are encoded for the Z3 SMT solver with LLVM's forms of undefined behavior. Loops are unrolled to a bound, which keeps checking fast but means some bugs are missed.

The tool was run over the functions in LLVM's unit-test suite that exercise IR-level transformations.

Results. The tool exposed 47 previously unknown bugs, of which 28 had been fixed at the time of writing, and led to eight corrections of the LLVM language reference.

Research gap. The approach depends on a full formal semantics of the IR and on a solver, which a small compiler does not have.

2) YARPGen

Solution. YARPGen [27] generates random C and C++ programs that are free of undefined behavior and uses them for differential testing of compilers.

Methodology and tools. The generator tracks values while generating and rewrites operations that could trigger undefined behavior; the rewrite rules were checked exhaustively on 4-bit values and with Z3.

Generation policies steer programs toward patterns that trigger specific optimizations. The compilers under test were GCC, LLVM and Intel's C++ compiler, all targeting x86-64.

Results. YARPGen found 83 new bugs in GCC, 62 in LLVM and 76 in the Intel compiler, and its policies triggered targeted optimizations 20% (LLVM) and 40% (GCC) more often.

Research gap. It tests production C compilers, not the compactness or losslessness of IR encodings.

E. CODE AND BYTECODE COMPRESSION

Compact program representations were studied intensively before the works above. Ernst *et al.* [28] derived two formats from the intermediate code of a C compiler: an interpretable format that can be executed without decompression and a denser wire format that is expanded before execution, and compared both with general-purpose compression of native code. Franz and Kistler [29] distributed programs as *slim binaries*, abstract syntax trees compressed with an adaptive dictionary from which native code is generated at load time. Pugh [30] reorganized the class files of a Java archive, sharing constants across classes and storing each kind of data in its own stream, before applying a general-purpose compressor. Lucco [31] split machine instructions into separate streams for opcodes, registers and immediates and compressed each with its own dictionary, which is the idea behind our columnar container. Evans and Fraser [32] inferred a grammar for bytecode from training programs and rewrote programs into specialized instructions, and Debray *et al.* [33] combined whole-program optimization with procedural abstraction to compact native binaries.

These works pursue the same goal as ours but mostly on native code or on full Java archives, and they do not separate the contribution of optimization from that of encoding. Among current formats, the WebAssembly binary format [2], [34] encodes a stack machine with structured control flow and stores integers as LEB128, as CLIR does. LLVM bitcode [35] is a bit-level stream with variable-width fields, abbreviations and relative value numbers, and the Java class file [36] combines a constant pool with stack-machine bytecode and fixed-width indices. Table 7 compares their design with CLIR, and Section IV-D4 compares their sizes.

F. SUMMARY OF THE RESEARCH GAP

The reviewed works either transform native code inside large production toolchains, learn optimization policies with substantial infrastructure, study bytecode without shrinking it, test compilers without looking at encodings, or compress code without separating optimization from encoding. None of them measures, at the level of a small IR, how much of a size reduction comes from optimization and how much from the binary format, or compares such a pipeline with general-purpose compressors applied to the *same* optimized program while checking that program behavior is preserved. The proposed work addresses this combination.

III. PROPOSED WORK AND SYSTEM

A. PROBLEM FORMULATION

Let P be a source program and $T = \langle t_1, \ldots, t_n \rangle$ the three-address code produced for it. For an input sequence I, the observable behavior $\mathrm{Obs}(T, I)$ is the sequence of printed values together with the returned value; we write $\mathrm{Obs}(T)$ when the input is fixed. The goal is to find an optimized instruction list T^* and an encoding E that minimize the encoded size while keeping the behavior and allowing the instructions to be decoded:

$$\begin{aligned} &\min_{T^*, E} \ |E(T^*)| \\ &\text{s.t.} \quad \mathrm{Obs}(T^*) = \mathrm{Obs}(T), \quad D\big(E(T^*)\big) \cong T^*, \end{aligned} \tag{1}$$

where D is the decoder and $\cong$ denotes equality up to a consistent renaming of variables and labels.

The decoder must need nothing but $E(T^*)$; in particular, no table kept by the encoder. The problem is split into two stages so that each can be measured on its own. The first

TABLE 1. Summary of the related works reviewed in Section II

Work	Technique	Tools and benchmarks	Reported result	Gap for compact IR
SalSSA [10]	Function merging in SSA form	LLVM 11; SPEC CPU2006/2017, MiBench	7.9–9.7% smaller code	Native code; no encoding
HyFM [11]	Block-level function merging	LLVM 11; SPEC CPU2006/2017	Merging 4.59–5.28× faster	Quadratic pairing; native
RoLAG [12]	Loop rolling by SSA graph alignment	LLVM 12; AnghaBench, SPEC, TSVC	9.12% on affected functions	Single block only
Inlining [13]	Size-reducing inlining heuristics	LLVM; MiBench	2.96% average, up to 11%	One suite, native
Optimal inlining [14]	Exhaustive study and autotuning	LLVM; SPEC2017, SQLite	7% average over LLVM	One transformation
iOS outlining [15]	Repeated whole-program outlining	Swift, LLVM, AArch64	23% smaller code section	Custom toolchain
MLGO [16]	Learned inlining policy	LLVM, TF-Agents	Up to 7% vs `-Oz`	Training cost
POSET-RL [17]	RL pass ordering	LLVM 10, IR2Vec; SPEC, MiBench	6.19% size, SPEC2017	Proxy rewards
LLM [18]	7B transformer picks passes	LLVM 10; 100k test functions	3.01% fewer instructions	No constant folding
CompilerGym [19]	Gym environments	LLVM, GCC, CUDA	27× faster steps	Platform only
ProGraML [20]	Program graphs for ML	LLVM, XLA; MPNN	Better downstream tasks	Learns exact analyses
Wasm study [24]	Empirical study of binaries	8461 binaries	29% of web binaries minified	No size reduction
Wasm superopt. [25]	Souper on LLVM to Wasm	12 Rosetta Code programs	0.33% median reduction	IR-level cost model
Alive2 [26]	Translation validation	Z3; LLVM unit tests	47 new bugs	Needs formal IR semantics
YARPGen [27]	UB-free random programs	GCC, LLVM, ICC	221 new bugs	Tests compilers, not encodings

stage reduces n to $n^* = |T^*|$ with semantics-preserving transformations on SSA form. The second stage chooses E among a compact bytecode, the same bytecode with canonical Huffman coding, and a column-oriented container. The reduction obtained by each stage is reported separately, and the constraint on Obs is checked by executing T and T^* with independent interpreters.

B. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture of the proposed system. The front end turns source text into tokens, an abstract syntax tree and TAC, and it fills a scoped symbol table and a constant pool. The middle end builds the control-flow graph (CFG), computes dominance information, converts the code to SSA form and optimizes it. The back end writes the optimized code in three encodings.

The three tiers communicate only through plain data: an instruction list, a CFG with basic blocks, and byte strings. This makes each tier testable on its own, and it lets the evaluation scripts of Section IV observe every intermediate result. A command-line driver and a web front end, which draws the CFG and the dominator tree, sit on top of the same core.

C. DETAILED METHODOLOGY

1) Front end and three-address code

The input language provides `int`, `float`, `string` and `bool` variables, arithmetic, relational and logical operators, `if`/`else`, `while` and `for` statements, nested blocks, a `print` statement, function definitions and calls (recursion included), and input with `input()` or `scanf`. Simple C programs are accepted as well: preprocessor lines are skipped, the body of `int main()` is the main program, `printf` is mapped to `print`, and `scanf("%d", &x)` reads one integer into `x`. Arrays and pointers are not supported. The TAC generator emits quadruples over 28 opcodes and loads every literal into a fresh temporary, so `int a = 10 + 20 * 3;` becomes six instructions. Every function other than the main program becomes its own unit, which starts with one `param` instruction per parameter; calls are `call` instructions and input is read by `read` instructions. All later stages process each unit separately. Temporaries are named so that they cannot coincide with a variable of the program.

2) Control-flow graph and dominance

Basic blocks start at the first instruction, at every jump target and after every jump or return; edges follow jumps, both outcomes of conditional jumps and fall-through. A block d dominates b if every path from the entry r to b passes through d. The dominator sets are the greatest fixed point of

$$\mathrm{Dom}(r) = \{r\}, \qquad \mathrm{Dom}(b) = \{b\} \cup \bigcap_{p \in \mathrm{pred}(b)} \mathrm{Dom}(p). \quad (2)$$

The immediate dominator of $b \neq r$ is the strict dominator that all other strict dominators of b dominate:

$$\mathrm{idom}(b) = d \in \mathrm{Dom}(b) \setminus \{b\} \quad \text{such that } \mathrm{Dom}(b) \setminus \{b\} \subseteq \mathrm{Dom}(d). \quad (3)$$

The dominance frontier of a block x collects the join points where the dominance of x ends:

$$\mathrm{DF}(x) = \{\, y \mid \exists p \in \mathrm{pred}(y) : \; x \in \mathrm{Dom}(p) \wedge x \notin \mathrm{Dom}(y) \setminus \{y\} \,\}. \quad (4)$$

The implementation computes DF by walking up the dominator tree from each predecessor of a join block until the immediate dominator of the join block is reached [5].

3) SSA construction

A variable receives ϕ-functions only if it is used in some block before being defined there, which yields semi-pruned SSA

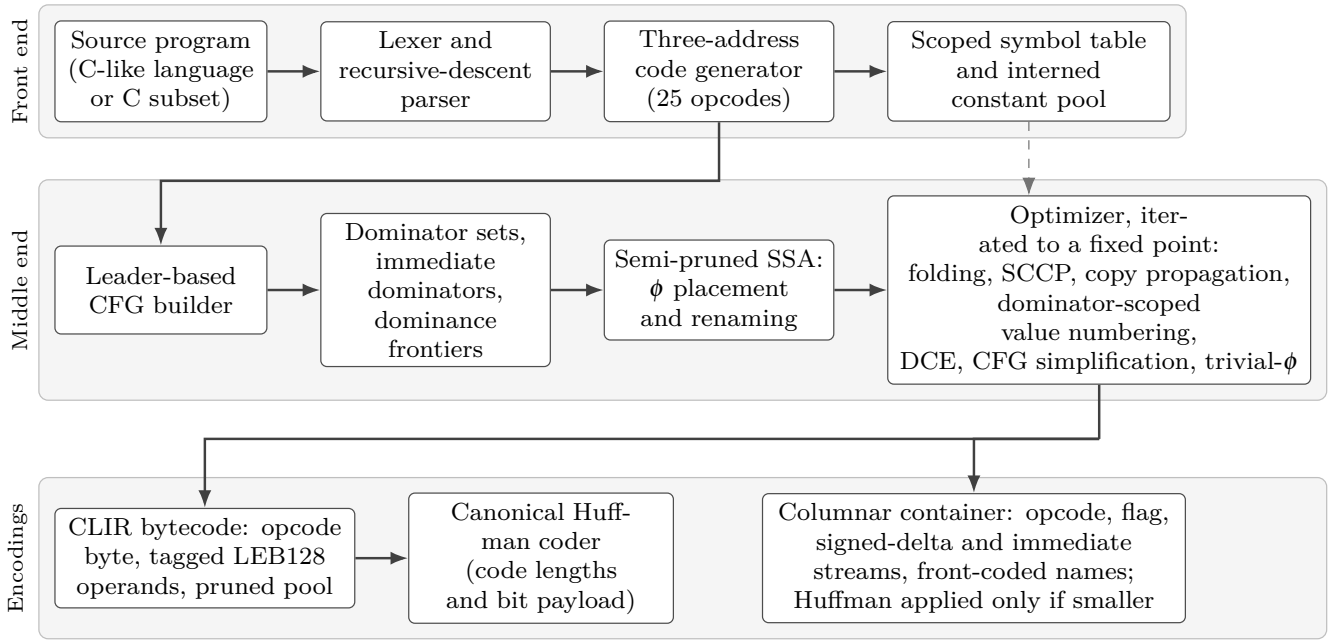


FIGURE 1. **Architecture of the proposed system (DCE: dead-code elimination). Solid arrows carry the program; the dashed arrow indicates that the constant pool and symbol names are also passed to the encoders.**

form. For such a variable v with definition blocks S_v, the ϕ-functions are placed on the iterated dominance frontier

$$\begin{aligned} \mathrm{DF}^{+}(S_v) &= \lim_{i\to\infty} \mathrm{DF}_i, \qquad \mathrm{DF}_1 = \mathrm{DF}(S_v), \\ \mathrm{DF}_{i+1} &= \mathrm{DF}(S_v \cup \mathrm{DF}_i). \end{aligned} \tag{5}$$

Variables are then renamed in a depth-first walk of the dominator tree, as listed in Algorithm 1. Fig. 2 shows the result for benchmark S2, which contains an `if/else` diamond followed by a loop: three ϕ-functions are inserted, one at the join of the diamond and two at the loop header.

4) Optimization passes

The optimizer applies seven passes in a loop until a round leaves the program unchanged or ten rounds have run (Algorithm 2). Constant folding rewrites an instruction whose operands are literals into a literal load,

$$\langle r \leftarrow a \odot b \rangle,\ a,b \in \mathcal{L} \implies \langle r \leftarrow \mathrm{eval}_{\odot}(a,b) \rangle, \tag{6}$$

where $\mathcal{L}$ is the set of literals and eval follows C: integer division truncates toward zero and the remainder takes the sign of the dividend,

$$a \,/\, b = \mathrm{sgn}(ab) \left\lfloor \frac{|a|}{|b|} \right\rfloor, \qquad a \,\%\, b = a - b\,(a \,/\, b). \tag{7}$$

Folding also applies the identities $x+0=x, x\cdot 1=x, x\cdot 0=0$ and $x-0=x$, replaces operands defined as literals, and turns a conditional jump on a literal into a jump or a fall-through, removing the dead CFG edge and its ϕ-arguments. It does not look through ϕ-functions.

Sparse conditional constant propagation [22] closes that gap. It assigns every SSA name x a value $\mathcal{V}(x)$ in the lattice

Algorithm 1 Semi-pruned SSA construction

Input: CFG G with dominator tree and frontiers DF
Output: G in SSA form

```
Glob ← variables used in a block before their definition there
for all v ∈ Glob do
    W ← S_v {blocks that define v}
    while W ≠ ∅ do
        remove a block x from W
        for all y ∈ DF(x) without a φ for v do
            insert v ← φ( ) after the labels of y
            if y ∉ S_v then add y to W
        end for
    end while
end for
Rename(r) {start at the entry block}
procedure Rename(b)
    give each φ and each definition in b a new version v_k and push it
    replace each use in b by the top of its stack
    fill the φ-arguments of every successor of b for edge (b, ·)
    Rename(c) for each child c of b in the dominator tree
    pop the versions pushed in b
```

$\top \sqsupset c \sqsupset \bot$ (undetermined, the constant c, not constant) and only follows CFG edges found executable; a ϕ-function takes the meet of the arguments on its executable incoming edges,

$$\mathcal{V}\big(x \leftarrow \phi(\ldots, p{:}a_p, \ldots)\big) = \bigsqcap_{(p,b)\ \text{executable}} \mathcal{V}(a_p), \tag{8}$$

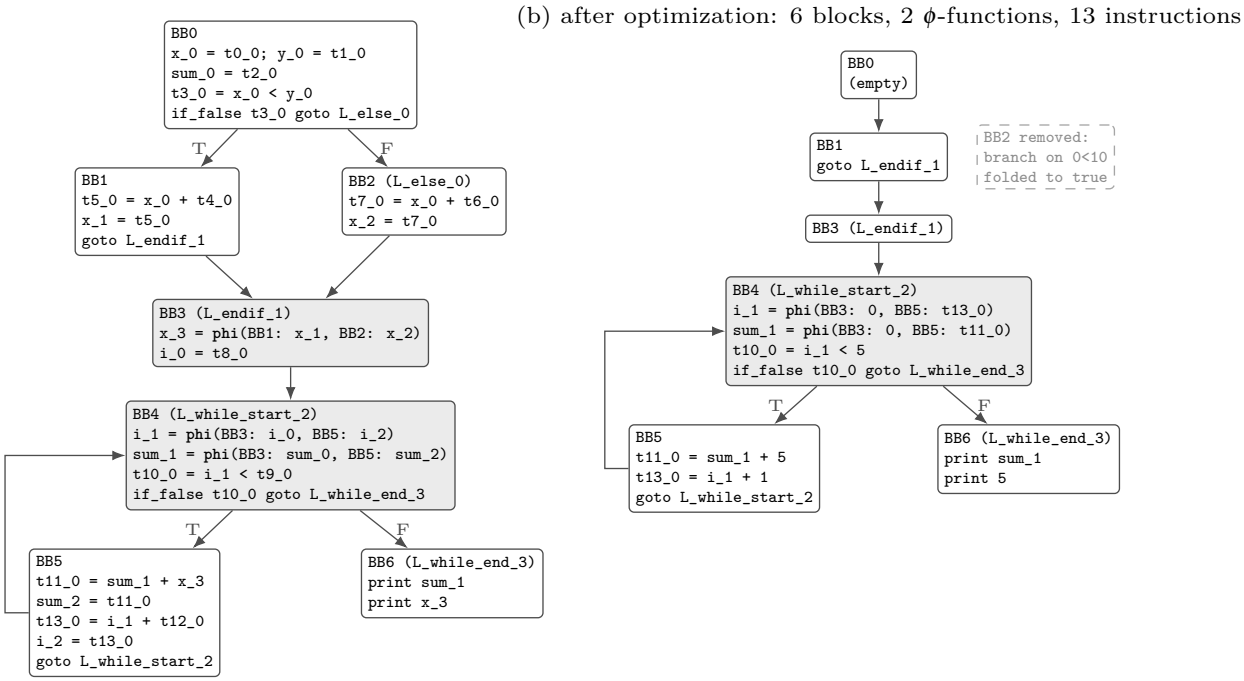


FIGURE 2. **Worked example for benchmark S2. (a) SSA form produced by Algorithm 1; literal loads into temporaries (for example `t4_0 = 5`) are omitted for space. (b) The same program after optimization: the branch on the constant condition $0 < 10$ is folded, BB2 becomes unreachable, the ϕ-function for `x` becomes trivial, and the value 5 is propagated into the loop and the final `print`.**

where $\top \sqcap v = v$, $c \sqcap c = c$, and every other meet gives $\bot$. Input, parameters and call results are $\bot$. Constant definitions become literal loads, constant uses become literals, a branch on a constant becomes a jump or a fall-through, and edges never found executable are removed. Copy propagation then replaces every use of x, where $x \leftarrow y$ is a copy, by y; in SSA form the definition of y dominates every use of x, so the copy becomes dead. Both passes run only on code in SSA form.

Value numbering walks the dominator tree with a scoped table indexed by the key

$$\kappa(i) = \begin{cases} (\mathrm{op}, \min(a,b), \max(a,b)) & \mathrm{op} \in \{+, \times, =, \neq\} \\ (\mathrm{op}, a, b) & \text{otherwise,} \end{cases} \tag{9}$$

and replaces a later instruction with the same key by a copy of the earlier result, which copy propagation removes in the next round; entries are withdrawn when the walk leaves a subtree, so values never flow between sibling branches. Dead-code elimination repeatedly removes every instruction i without side effects whose result is not used,

$$\mathrm{def}(i) \notin \bigcup_j \mathrm{uses}(j) \setminus \{\mathrm{def}(j) \mid j \text{ is a } \phi\} \;\Rightarrow\; \text{delete } i, \tag{10}$$

where a ϕ-function's use of its own result does not keep it alive; `print`, `read`, `call`, jumps and returns are never removed. CFG simplification deletes unreachable blocks and stale ϕ-arguments, and trivial-ϕ removal turns a ϕ-function whose other arguments all name one value into a copy.

Algorithm 2 Fixed-point driver of the optimizer

Input: CFG G of one function in SSA form
Output: optimized CFG G

for $k = 1$ **to** 10 **do**
  $G_0 \leftarrow$ copy of G
  FoldAndPropagate(G) {(6), (7)}
  SCCP(G) {(8); may delete edges}
  PropagateCopies(G)
  ValueNumber(G) {(9); dominator-tree walk}
  EliminateDeadCode(G) {(10)}
  PruneUnreachable(G)
  RemoveTrivialPhis(G)
  if $G = G_0$ **then**
    break
  end if
end for
return G

5) Encodings

The optimized blocks of every function are linearized in order and written as one CLIR package, whose layout is drawn in Fig. 3(a). Each instruction is one opcode byte followed by its operands as unsigned LEB128 integers, which store seven payload bits per byte. An operand v whose binary form has $\beta(v)$ significant bits, with $\beta(0) = 0$, therefore occupies

$$\ell(v) = \max(1, \lceil \beta(v)/7 \rceil) \text{ bytes.} \tag{11}$$

Variables, labels and blocks are numbered per function in order of first appearance. A value operand is tagged in its lowest bit, $\tau(x) = 2r$ for a variable in register r and $\tau(c) = 2q + 1$ for the literal at index q of the constant pool, so literals that propagation places into instructions are part of the package. The pool is built from the optimized code and holds exactly the literals it refers to, in order of first use; an integer is stored as a zigzag-mapped LEB128 value, a Boolean as a tag byte alone. A ϕ-argument names its predecessor by block ordinal, and the body lists the blocks of each function, so a decoder can rebuild every CFG and execute the program. With f functions, block counts b_g, instruction counts m_B and pool entries of e_j bytes, the package size is

$$\begin{aligned} |CLIR| &= 5 + \ell(k) + \textstyle\sum_{j=1}^{k} e_j + \ell(f) \\ &\quad + \textstyle\sum_{g=1}^{f} \Big(\ell(b_g) + \sum_{B} \big(\ell(m_B) + body_B \big) \Big), \qquad (12) \\ body_B &= \textstyle\sum_{i \in B} \Big(1 + \sum_{j=1}^{k_i} \ell(\tau(v_{i,j})) \Big). \end{aligned}$$

The canonical Huffman stage (Fig. 3(b)) computes a Huffman code for the k distinct byte values of the package, limited to 15 bits, and stores only the code length of each value; the decoder rebuilds the canonical code [23], in which codes of equal length are consecutive integers in symbol order. The header holds the input length n as LEB128 and the smallest of three length tables: a list of the k symbols with 4-bit lengths, a 32-byte bitmap with 4-bit lengths, or 256 4-bit lengths. For a byte value s of frequency f_s and code length ℓ_s,

$$\begin{aligned} h &= \ell(n) + \min\big(2 + k + \lceil k/2 \rceil,\ 33 + \lceil k/2 \rceil,\ 129\big), \\ |H| &= h + \left\lceil \frac{1}{8} \sum_{s} f_s \, \ell_s \right\rceil, \qquad \textstyle\sum_s f_s \ell_s \geq n\,\mathcal{H}, \end{aligned} \qquad (13)$$

where h is the header and $\mathcal{H}$ the empirical entropy of the input in bits per byte [9]. The header never exceeds 132 bytes. The first release stored a 32-bit frequency for each symbol instead, $h = 7 + 5k$ bytes, up to 1287 bytes; Section IV reports both. Writing the payload as about $n\mathcal{H}/8$ bytes, coding saves space only when $n(1 - \mathcal{H}/8)$ exceeds the header, which gives the break-even size

$$n^* \approx \frac{h}{1 - \mathcal{H}/8}. \qquad (14)$$

The break-even point is therefore a property of the header design, not of Huffman coding as such.

The columnar container (Fig. 3(c)) splits the instructions of each function into an opcode stream, a flag stream, a stream of distances from each use to its definition, and a stream of literals and labels stored as strings, followed by a front-coded table of source names. A definition is implied by its position i, and a use of a value defined at position d stores $\delta = i - d$ in zigzag form, 2δ for $\delta \geq 0$ and $-2\delta - 1$ otherwise; a loop-carried ϕ-argument is defined later in the linear order and has $\delta < 0$. Each ϕ-argument and call argument carries its own kind flag. A leading flag byte records whether Huffman coding was applied, which is done only when it makes the container smaller. Algorithm 3 summarizes the encoding stage.

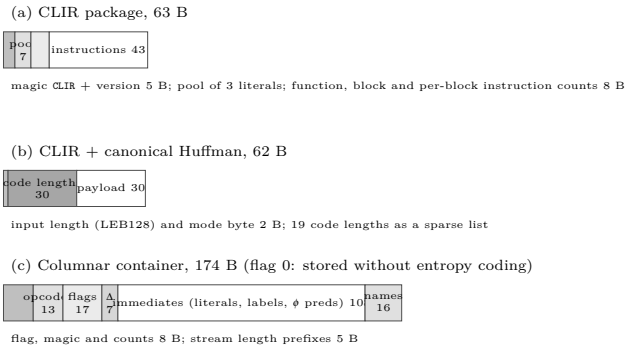


FIGURE 3. Byte layout of the three encodings of benchmark S2, drawn to scale. The sizes are those measured in Section IV.

Algorithm 3 Encoding with adaptive entropy coding

Input: optimized functions $T_1^*, \ldots, T_f^*$ (main first), names N
Output: CLIR package B, Huffman package H, container Q

$C \leftarrow$ empty constant pool; $body \leftarrow$ LEB128(f)
for all functions T_g^* **do**
  number its variables, labels and blocks from 0
  append LEB128 of its block count to $body$
  for all blocks B of T_g^*, instructions $t \in B$ **do**
    append the opcode byte of t and τ of its operands, adding new literals to C
  end for
end for
$B \leftarrow$ "CLIR" $\|$ version $\|$ C $\|$ $body$
assert decoding B alone gives $T_1^*, \ldots, T_f^*$ up to renaming
$H \leftarrow$ CanonicalHuffman(B); **assert** Decode$(H) = B$
$S \leftarrow$ per-function opcode, flag, delta and immediate streams and front-coded N
$h \leftarrow$ CanonicalHuffman(S); $Q \leftarrow$ 0x01$\|h$ **if** $|h| < |S|$ **else** 0x00$\|S$
assert decoding Q gives $T_1^*, \ldots, T_f^*$ up to renaming
return B, H, Q

D. MODULES AND THEIR FUNCTIONS

Fig. 4 lists the modules of the implementation and their main functions. The front end consists of the lexer, the parser, the TAC generator and the symbol and constant tables. The middle end consists of the CFG builder, the dominance analyzer, the SSA transformer and the optimizer. The back end holds the two encoding modules, and the interface layer consists of the command-line driver, an HTTP server with three endpoints and the browser front end. The evaluation scripts in the directory `paper_eval` (Section IV) only read these modules and never modify them.

The work on these modules was divided as follows. Rishabh Kumar implemented the first version of the compiler core (front end, control-flow graph, dominance analysis, SSA construction, optimizer and encoders), the HTTP server and the web front end with its user-interface tests. Hrituraj Deb led the revision: the corrected constant folder, SCCP

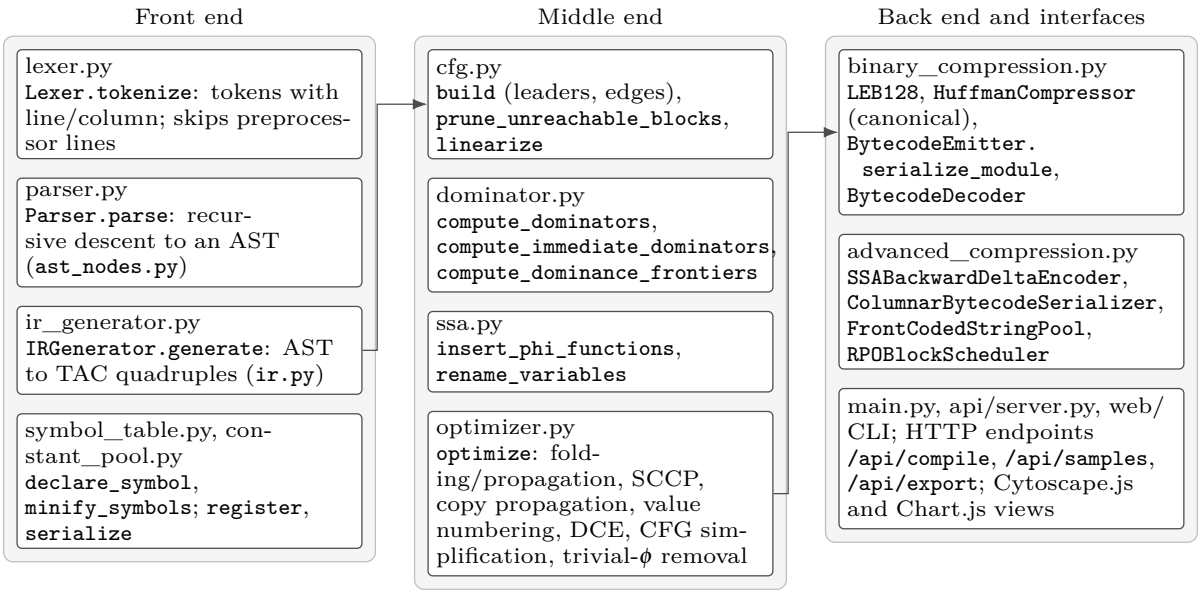


FIGURE 4. Modules of the implementation and their main functions.

and copy propagation, the self-describing bytecode, canonical Huffman coding, function and input support, the kernels, the evaluation and the manuscript. Gautham G and Aarnav Bhasin wrote and ran test programs, checked the benchmark outputs, and prepared the diagrams and the screenshot. Every module was reviewed by at least one other team member.

E. FLOW DIAGRAM

Fig. 5 shows the processing flow for one program. A syntax error stops processing with the line and column of the offending token. Otherwise the program passes through TAC generation, CFG construction, dominance analysis and SSA construction. The optimizer then repeats its seven passes while the program keeps changing, for at most ten rounds. Every function of the program passes through these stages on its own. Finally, the encoder writes the CLIR package and its Huffman-coded form, builds the columnar container, decides with one comparison whether that container is stored coded or raw, and verifies every round trip before reporting sizes.

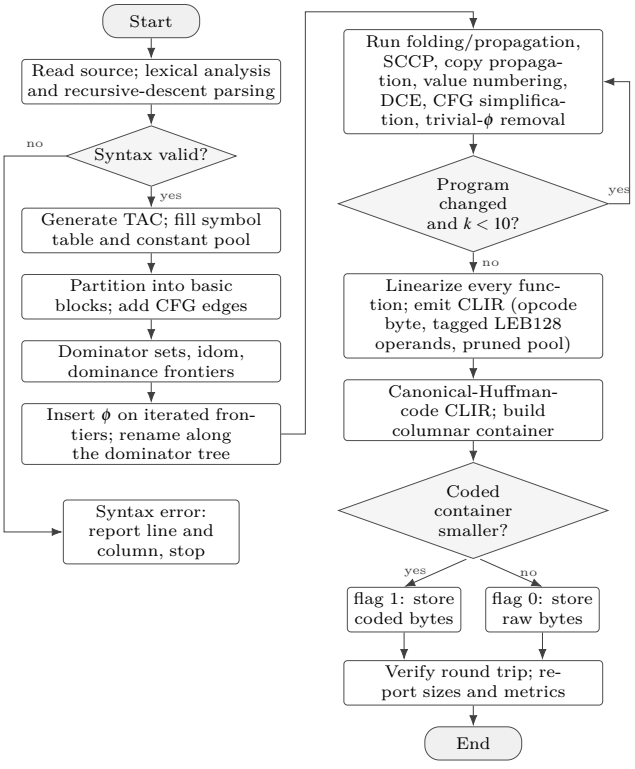


FIGURE 5. Processing flow of the proposed system for one input program.

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. TOOLS, CONFIGURATION AND SETUP

The system is written in Python and runs on the Python 3 standard library alone [37]; the web front end uses Cytoscape.js to draw graphs [38]. All measurements were taken with Python 3.13.12 on one machine (macOS 26.6.2, arm64). The general-purpose compressors were called through Python: Brotli 1.2.0, Zstandard 1.5.7, zlib 1.3.1 for DEFLATE, and the standard `lzma` module for xz. The existing bytecode formats were produced with Apple clang version 17.0.0 (clang-1700.6.3.2) and javac 24.0.2. A compilation is started with `python3 main.py --input benchmarks/<file>`, and the web front end with `python3 main.py --web --port 8000`.

The evaluation scripts are released in the directory `paper_eval` of the repository; `python3 paper_eval/run_all.py` runs `measure`, `difftest`, `ablation`, `scaling`, `compare`, `baselines`, `make_tables` and

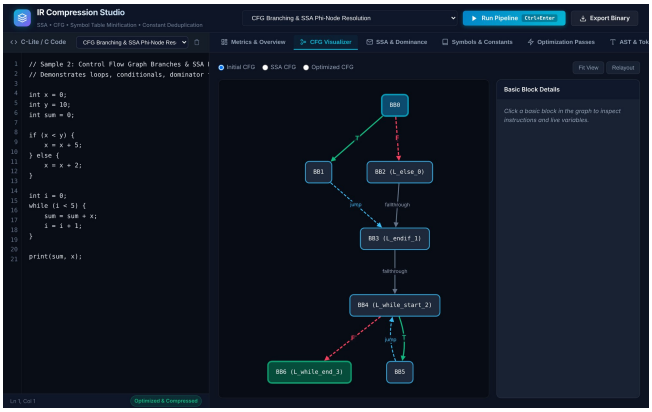


FIGURE 6. Screenshot of the implementation: the web front end showing the source of benchmark S2 and its initial control-flow graph with true (T), false (F), jump and fall-through edges.

`make_figures` in sequence and regenerates every table, every data figure and every number quoted in this article. The scripts cross-check the pipeline in five ways: every CLIR package and every columnar container is decoded without any table from the encoder and compared with the optimized program; every CLIR package is also rebuilt into control-flow graphs and executed; every Huffman size is predicted with (13) and every stream decoded; program behavior is compared by two reference interpreters, one for linear TAC and one for SSA graphs, which follow (7); and the kernels are compiled natively and translated to Java, and both must print the same numbers as our pipeline. The repository also contains 113 unit and regression tests, including a small differential test; the 110 that need only the standard library all pass, and the other three exercise optional compressor bindings of the web front end. Fig. 6 shows the web front end after compiling benchmark S2.

B. PARAMETER SETUP

Table 2 lists the parameters of the experiments, and Table 3 describes the benchmarks. The seven demonstration programs S1–S7 were written to show the individual passes and have no input. The ten kernels K1–K10 were written for this revision after well-known small C programs: Euclid's algorithm, Newton's integer square root, modular exponentiation, bit counting by repeated division (in the spirit of the MiBench *bitcount* kernel), Collatz sequences, trial-division primality, Fibonacci numbers computed iteratively and recursively, the running sums of Adler-32, Zeller's congruence and a fixed-point low-pass filter. Each kernel reads 1 to 13 input values with `scanf` and calls one or two functions, so its values are not known at compile time. The kernels are valid C, and their output compiled natively with clang is the reference for our pipeline. The random programs come from a seeded generator that declares integer variables, assigns expressions with $+$, $-$, $\times$, $/$ and $\%$, and nests `if`/`else` statements and counter-controlled loops. Every variable is initialized and has a unique name, divisors are nonzero literals and loops run at most four times, so every program terminates without a run-time error. In the variant with input, 30% of the declarations read their initial value with `input()`.

TABLE 2. Parameter setup

Parameter	Value
Benchmarks	7 demonstration programs (S1–S7), 326–539 B of source, no input; 10 kernels (K1–K10), 551–901 B, with input and function calls
Random programs	1,000 + 500 with input (differential testing), 200 + 200 with input (ablation), 64 with input (scaling: 8 size classes $\times$ 8 seeds), 300 (dictionary training); disjoint seeds
Program sizes	4 to 48 top-level statements (4 to 512 for scaling); loops of 1–4 iterations; nesting up to 3 levels
Optimizer	7 passes, at most 10 rounds
Timing	median of 30 runs (benchmarks); minimum of 3 runs (random programs)
Huffman coding	canonical, code lengths of at most 15 bits; frequency-table header of the first release for comparison
Brotli	quality 11, window 2^{22}
Zstandard	level 19; standard frame, or magic-less frame without checksum, content size and dictionary id
xz	preset 9; `.xz` container, or raw LZMA2 stream
DEFLATE	level 9, raw stream
Shared dictionary	4,096 B, trained by Zstandard on the 300 training programs, per representation; also used as DEFLATE preset dictionary
Existing formats	LLVM bitcode: `clang -Oz -g0 -emit-llvm`; Java class files: `javac -g:none`

TABLE 3. Benchmarks: source size, number of functions (Fn, including the main program) and input values (In); instructions, blocks, edges and ϕ-functions refer to the code before optimization, summed over all functions.

ID	Focus	Source (B)	Fn	In	TAC instr.	Blocks	Edges	ϕ
S1	Constant folding, identities	419	1	0	33	1	0	0
S2	Diamond + loop, ϕ nodes	326	1	0	32	7	8	3
S3	Redundant subexpressions	419	1	0	32	4	4	0
S4	Dead code, constant branch	410	1	0	28	4	4	1
S5	Fibonacci loop	392	1	0	27	4	4	3
S6	Diamond + loop (CFG demo)	473	1	0	24	7	8	2
S7	C program (`main`, `printf`)	539	1	0	27	7	8	2
K1	GCD and LCM (Euclid)	551	3	9	42	9	8	3
K2	Integer sqrt (Newton)	657	3	6	57	13	12	3
K3	Modular exponentiation	716	3	10	56	11	11	5
K4	Bit counting	795	3	7	70	12	12	6
K5	Collatz lengths	726	2	7	62	13	15	10
K6	Primes by trial division	685	2	1	66	14	15	8
K7	Fibonacci (iter., rec.)	629	3	2	58	10	9	4
K8	Adler-32 running sums	554	2	13	34	5	4	3
K9	Day of week (Zeller)	757	3	13	84	10	9	3
K10	Low-pass filter	901	3	11	64	14	14	6

C. PERFORMANCE METRICS

Six metrics are used. For a quantity that falls from a to b, the reduction is

$$R(a,b) = 100\,\frac{a-b}{a}\ \%, \tag{15}$$

which gives the instruction reduction when applied to instruction counts and the size reduction when applied to bytes. The encoding density is the number of bits spent per instruction,

$$\rho = \frac{8\,|X|}{N}, \tag{16}$$

for a representation X of N instructions. The compilation throughput is $\Theta = n/t$ instructions per second for a program of n raw instructions compiled in time t, and the entropy-coding overhead is the share $h/|H|$ of the header in (13). Correctness is measured by the agreement rate

$$A = 100\,\frac{|\{P:\ \mathrm{Obs}(T_P^*, I_P) = \mathrm{Obs}(T_P, I_P)\}|}{|\{P\}|}\ \% \tag{17}$$

over a set of programs P with inputs I_P. *Raw TAC text* denotes the unoptimized instructions, one per line, in UTF-8, with a header line `func f(a, b):` before each function other than the main program; *optimized text* is the optimized program printed the same way. The optimized text and the CLIR package encode the same optimized program, so comparing compressors on the optimized text with CLIR isolates the effect of the encoding.

D. RESULTS

1) Instruction reduction

Table 4 and Fig. 7 show the instruction counts before and after optimization. With all seven passes, the demonstration programs lose 48.1% (S5) to 89.3% (S4) of their instructions, 66.6% on average, but the kernels only 19.0% to 34.3%, 28.3% on average. The demonstration programs have no input, so folding and SCCP turn almost every value into a literal and dead-code elimination removes the literal loads. In the kernels, values that depend on input, parameters and call results are unknown at compile time; the passes mainly remove the literal temporaries and copies that the TAC generator produces. The reductions of the demonstration programs are therefore an upper bound that programs with input do not reach. The five passes of the first release reach 18.5% to 89.3% and 7.1% to 26.2%. The new passes help most where values flow around loops: S5 keeps 22 of its 27 instructions with the five passes and 14 with all seven, because copy propagation removes the copies that carry the Fibonacci recurrence from one iteration to the next. Value numbering found nothing to replace in any benchmark, since folding had already turned the repeated expressions of S3 into constants.

Fig. 8 explains where the reduction comes from, using 200 random programs without input and 200 that read input, with one pass disabled at a time. Without input, the 64,048 raw instructions fall to 25,173 with all passes (60.7%) and to 32,165 with the five passes of the first release (49.8%). Disabling dead-code elimination increases the result by 151.9% and disabling copy propagation by 17.4%; folding and SCCP overlap, so disabling either alone costs only 1.6% and 0.8%, while adding SCCP alone to the five base passes leaves 29,550 instructions and adding copy propagation alone 25,386. Value numbering replaced 108 expressions in 67 programs; with copy propagation the copies it creates disappear, and disabling it now costs 66 instructions, whereas in the first release it removed almost nothing. With input the picture is the same (60,493 to 26,749 instructions, 55.8%), because only some declarations read input. The top bars show the unoptimized forms relative to the fully optimized result; relative to the raw TAC, SSA conversion itself adds only the ϕ-functions, 8.9% more instructions without input (0.0% to 16.1% on the benchmarks).

2) Size reduction

Table 5 and Fig. 9 give the size of every representation. The CLIR package is 79.3% to 94.4% smaller than the raw TAC text on the demonstration programs and 74.2% to 78.7% on the kernels, and 27.3% to 75.0% and 75.7% to 79.8% smaller than the optimized text. These sizes include everything a decoder needs: literal operands are stored in the constant pool, the pool contains only the literals the code refers to, and the block and function structure is explicit. In the first version of this article, literal operands were register indices whose values were not written, and the pool was written as built by the front end; S4 then took 61 bytes, 47 of them for a pool that no instruction used. It now takes 20 bytes, of which 4 are its pool.

The optimized text is not always smaller than the raw text; it is larger for 9 of the 17 benchmarks (K1, K2, K3, K4, K5, K6, K7, K8, K10). SSA conversion appends a version number to every variable occurrence (`x` becomes `x_0`) and adds ϕ-lines such as `i_1 = phi(BB3: 0, BB5: t13_0)`, so the SSA text is 29.8% to 68.9% longer than the raw text, although it has only 0.0% to 16.1% more instructions. In the kernels, version suffixes take 1,360 bytes and ϕ-lines 1,626 bytes of the optimized text, which is 8,401 bytes in total against 7,818 for the raw text; the 171 instructions that optimization removes save less than that. The optimized-text column therefore reflects the spelling of SSA names, not the amount of code; CLIR numbers names and is not affected.

3) Comparison with general-purpose compressors

To isolate the effect of the encoding, we compressed the optimized text, which describes the same optimized program as the CLIR package, with three widely used general-purpose compressors: Brotli [39], Zstandard [40] and xz [41]. Table 6 and Fig. 10 show that CLIR is smaller than the best of the three on every benchmark: by 3.4% to 59.7% on the demonstration programs and by 39.5% to 50.9% on the kernels. The smallest margin is S1, whose package consists mainly of string literals. Against the raw text, the baseline of the first version of this article, the margins are 58.8% to 89.8% and 44.4% to 53.9%; they mix the effect of optimization with that of encoding and are not used for the comparison.

For inputs of a few hundred bytes, container headers are a large share of the output. Without them, that is, with Zstandard frames that omit the magic number, checksum, content size and dictionary id, and with raw LZMA2 instead of the `.xz` format (Brotli streams and raw DEFLATE have no

TABLE 4. Effect of optimization on the benchmarks: instruction counts with the five passes of the first release (Base) and with all seven passes (All), reduction R (%) with all passes, blocks after optimization, and the work done by each pass with all passes enabled (SCCP: operands and definitions replaced by constants; VN: value numbering).

ID	TAC	Opt. (Base)	Opt. (All)	Red. (%)	Blocks	Folds	SCCP	Copies	Dead instr.	Dead blocks	VN hits
S1	33	6	6	81.8	1	7	9	0	27	0	0
S2	32	17	13	59.4	6	4	10	2	20	1	0
S3	32	7	7	78.1	3	10	7	0	22	1	0
S4	28	3	3	89.3	3	5	9	0	23	1	0
S5	27	22	14	48.1	4	0	8	6	16	0	0
S6	24	13	11	54.2	6	2	8	1	13	1	0
S7	27	14	12	55.6	6	2	7	1	15	1	0
K1	42	39	34	19.0	9	0	2	4	11	0	0
K2	57	50	43	24.6	13	0	2	11	17	0	0
K3	56	52	45	19.6	11	0	2	6	16	0	0
K4	70	57	46	34.3	12	1	8	9	30	0	0
K5	62	53	42	32.3	13	1	8	10	30	0	0
K6	66	56	46	30.3	14	0	10	5	28	0	0
K7	58	49	40	31.0	10	0	10	4	22	0	0
K8	34	29	23	32.4	5	0	6	4	14	0	0
K9	84	62	56	33.3	10	0	2	7	31	0	0
K10	64	57	47	26.6	14	0	11	9	23	0	0

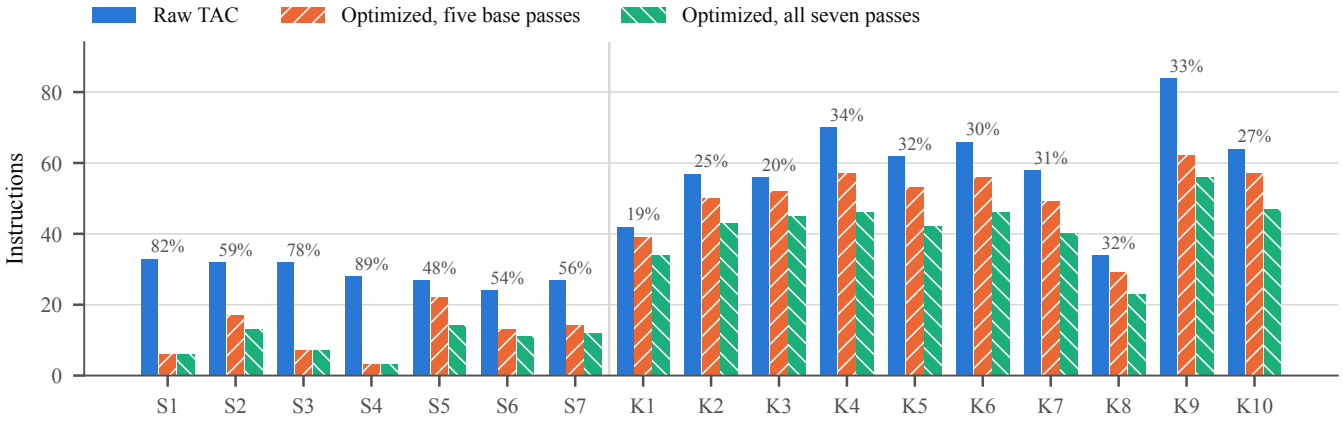


FIGURE 7. Instructions before and after optimization, with the five passes of the first release and with all seven passes; the label gives the reduction R with all passes. The vertical line separates the demonstration programs from the kernels.

TABLE 5. Encoded size in bytes. CLIR is shown as total (constant pool/instruction section), canonical Huffman as total (header/payload). R is the reduction of CLIR relative to the raw and to the optimized text. The frequency-table column is the Huffman coder of the first release. The entropy bound is $n\mathcal{H}/8$ for the CLIR package. "Columnar" includes its one-byte flag.

ID	Raw TAC text	Optimized text	CLIR (pool/instr.)	R vs. raw text (%)	R vs. opt. text (%)	CLIR+canonical Huffman (hdr/payload)	CLIR+freq.-table Huffman	Entropy bound	Columnar
S1	376	77	56 (36/15)	85.1	27.3	85 (51/34)	206	33	107
S2	360	244	63 (7/51)	82.5	74.2	62 (32/30)	132	29	174
S3	360	70	29 (5/19)	91.9	58.6	39 (26/13)	95	13	96
S4	360	35	20 (4/11)	94.4	42.9	30 (21/9)	76	8	91
S5	350	280	70 (9/56)	80.0	75.0	69 (35/34)	146	33	192
S6	317	205	52 (7/40)	83.6	74.6	54 (30/24)	121	23	159
S7	391	240	81 (34/42)	79.3	66.2	100 (52/48)	235	47	196
K1	555	635	141 (10/126)	74.6	77.8	122 (49/73)	230	72	282
K2	761	815	174 (12/157)	77.1	78.7	145 (52/93)	265	92	349
K3	817	964	195 (12/178)	76.1	79.8	160 (52/108)	285	107	366
K4	891	900	190 (14/171)	78.7	78.9	151 (51/100)	262	99	403
K5	818	915	189 (16/168)	76.9	79.3	154 (51/103)	265	102	399
K6	847	943	197 (12/180)	76.7	79.1	159 (52/107)	279	106	408
K7	753	778	170 (12/153)	77.4	78.1	136 (48/88)	240	87	321
K8	411	436	106 (14/87)	74.2	75.7	109 (51/58)	230	58	225
K9	1059	1039	243 (28/210)	77.1	76.6	201 (57/144)	371	143	351
K10	906	976	212 (28/179)	76.6	78.3	179 (56/123)	335	122	396

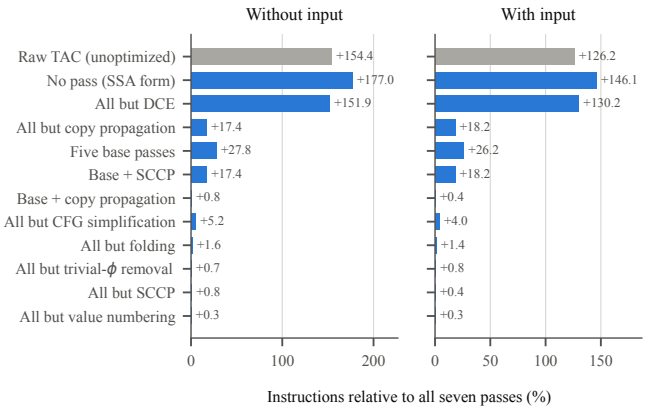


FIGURE 8. Per-pass ablation over 200 random programs without input and 200 that read input: total instructions after optimization with a pass disabled, with the five base passes, with base plus one new pass, and with no pass at all (the SSA form), relative to all seven passes. The gray bar is the raw TAC.

container to remove), CLIR is still 39.5% to 50.9% smaller on the kernels and smaller on 6 of the seven demonstration programs; the exception is S1. Many small files of the same kind are best compressed with a shared dictionary. With a 4,096-byte dictionary trained on 300 random programs and not counted per file, Zstandard or DEFLATE beat CLIR on S1, S2, S3, S4, S6. These are the smallest programs, whose optimized text consists almost entirely of tokens the dictionary contains. On all kernels, CLIR remains 25.5% to 37.9% smaller.

The two stages remove different kinds of redundancy. A compressor finds repeated byte strings but must keep every instruction; the optimizer removes instructions, and CLIR replaces names and literals by small indices. Both can be combined. On the 64 random programs of the scaling study, CLIR alone was smaller than the best compressor applied to the optimized text in 46 cases (from −10.3% to 58.3%); in the largest programs the compressors find enough repetition to win. Brotli applied to the CLIR package was smaller than the best compressor applied to the optimized text in 63 of the 64 programs.

4) Comparison with existing bytecode formats

Table 7 compares the design of CLIR with the WebAssembly binary format [2], LLVM bitcode [35] and the Java class file [36], and Table 8 gives the size of the kernels in the last two formats; the installed compiler had no WebAssembly target. Java class files are 3.2 to 6.0 times larger than CLIR, or 2.0 to 3.5 times after subtracting the size of an empty program in each format (311 and 13 bytes). LLVM bitcode is 22 to 46 times larger, or 11 to 24 times net of an empty module of 2,672 bytes. The comparison is not between equals: both formats carry types, symbol names and metadata that are needed for verification, linking and code generation and that CLIR leaves out, and `javac` does not optimize. It shows that a format's fixed per-file cost dominates for programs of this size. All ten native builds and Java translations printed the same values as our pipeline.

5) Encoding density

Fig. 11 expresses the sizes per instruction with (16). The raw TAC text spends 90.0 to 115.9 bits per instruction on the demonstration programs and 96.7 to 116.7 on the kernels. The CLIR instruction section spends 20.0 to 32.0 and 29.2 to 32.0 bits per optimized instruction: 16 bits is one opcode byte plus one operand byte, and binary operations, ϕ-functions and the block and function counts add more. Counted against the original instructions, the whole CLIR package costs 5.7 to 24.0 and 21.7 to 27.9 bits per raw instruction.

6) Entropy-coding overhead

Canonical Huffman coding made 11 of the 17 packages smaller, including 9 of the ten kernels, by up to 20.5%; its header is 28.4% to 70.0% of the output (Table 5). With the frequency-table header of the first release, every package became 1.4 to 3.8 times larger. Fig. 12 shows where entropy coding starts to help. Across the 64 random programs of the scaling study, canonical coding was no smaller than its input for every package of up to 81 bytes and smaller for every package of 115 bytes or more; with the frequency table, the threshold lay between 5,031 and 7,837 bytes. In 24 of the programs all 256 byte values occur, so the canonical header takes 132 bytes against 1287 for the frequency table, and the entropy of the packages larger than 1 kB lies between 6.06 and 6.51 bits per byte. Applied program by program, (14) classifies 64 of the 64 programs correctly for the canonical header and 64 for the frequency table. The break-even point of about 5.7 kB reported in the first version of this article was thus caused by the header, not by Huffman coding. In the largest size class, with a median of 6,117 raw instructions, canonical coding saved 18.4%, the frequency-table coder 12.1%, DEFLATE 20.8% and Brotli 31.3%: an order-0 code cannot exploit the repeated instruction sequences that dictionary coders find. The columnar container was larger than CLIR on all 17 benchmarks and all 64 random programs, because it stores literals, labels and ϕ-predecessors as strings and carries the names of the source variables.

7) Latency and throughput

Table 9 breaks the compilation time of each benchmark down by stage. The complete pipeline takes between 1.01 and 3.96 ms per benchmark, including both encoders. The optimizer is the most expensive stage in 17 of the 17 benchmarks, with 18.9% to 36.9% of the time. Fig. 13 shows the time for all 64 random programs. A least-squares fit on the logarithms gives an exponent of 1.15, so growth is slightly faster than linear; every round of the optimizer recomputes the dominator tree for value numbering with the iterative fixed point of (2), whose cost grows faster than linearly. The throughput ranges from 10,726 to 34,784 raw instructions per second.

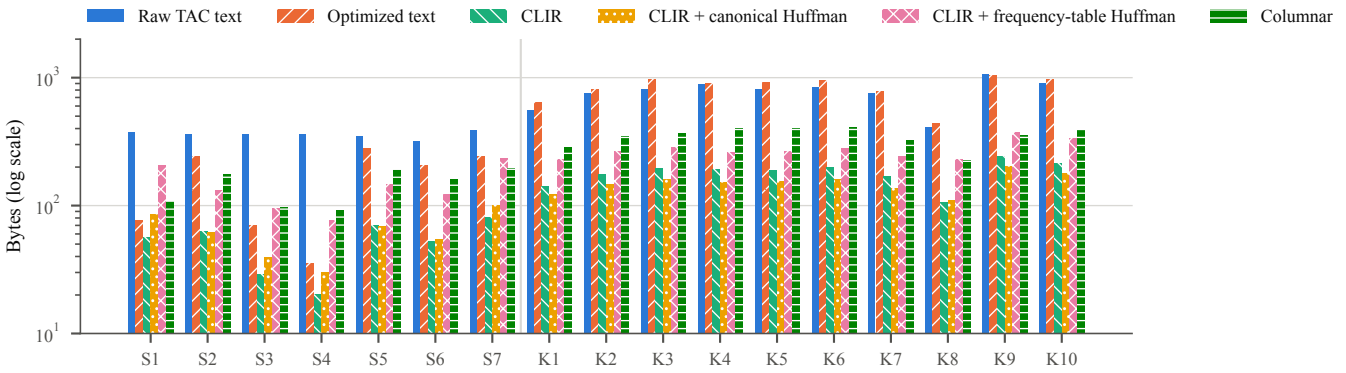


FIGURE 9. Encoded size of the benchmarks (logarithmic scale). Every value is listed in Table 5.

TABLE 6. Comparison with general-purpose compressors applied to the optimized text, which encodes the same optimized program as CLIR (bytes). Headerless: magic-less Zstandard frame and raw LZMA2 instead of the containers. Dictionary: Zstandard or DEFLATE with a shared 4,096-byte dictionary. *R* is the reduction of CLIR relative to the best compressor in each setting; a negative *R* means that the compressor is smaller.

ID	Optimized text	Brotli [39]	Zstd [40]	xz [41]	Best headerless	Best with dictionary	CLIR	*R* framed (%)	*R* headerless (%)	*R* dictionary (%)
S1	77	63	58	108	54	41	56	3.4	−3.7	−36.6
S2	244	147	147	204	143	55	63	57.1	55.9	−14.5
S3	70	52	51	100	47	18	29	43.1	38.3	−61.1
S4	35	39	41	96	37	9	20	48.7	45.9	−122.2
S5	280	155	156	216	151	84	70	54.8	53.6	16.7
S6	205	129	133	192	129	49	52	59.7	59.7	−6.1
S7	240	159	157	224	153	81	81	48.4	47.1	0.0
K1	635	264	289	356	264	200	141	46.6	46.6	29.5
K2	815	336	355	424	336	263	174	48.2	48.2	33.8
K3	964	382	398	476	382	314	195	49.0	49.0	37.9
K4	900	314	349	416	314	255	190	39.5	39.5	25.5
K5	915	360	382	452	360	278	189	47.5	47.5	32.0
K6	943	357	379	456	357	292	197	44.8	44.8	32.5
K7	778	304	326	392	304	237	170	44.1	44.1	28.3
K8	436	216	230	296	216	159	106	50.9	50.9	33.3
K9	1039	414	432	500	414	354	243	41.3	41.3	31.4
K10	976	379	398	460	379	323	212	44.1	44.1	34.4

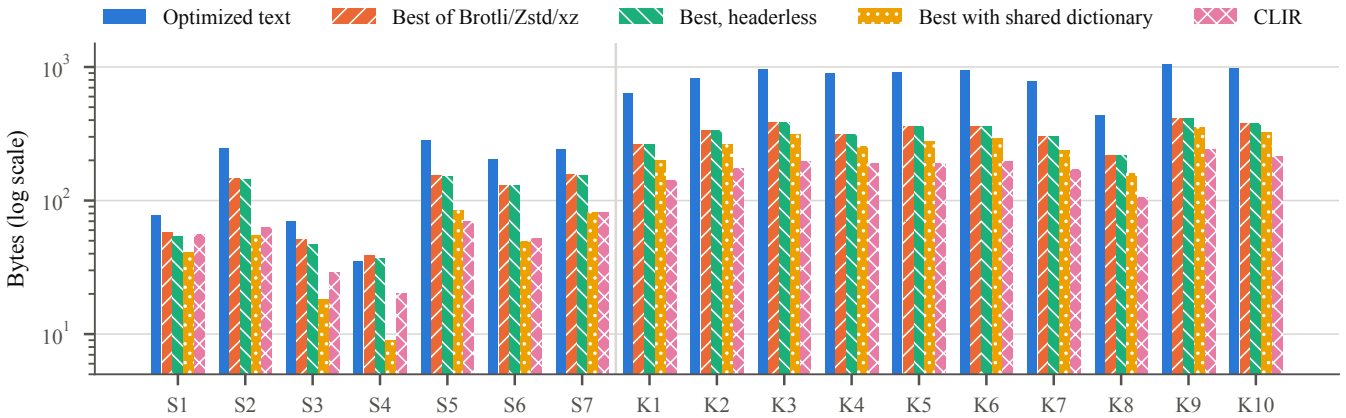


FIGURE 10. Comparison with general-purpose compressors applied to the optimized text (logarithmic scale). Every value is listed in Table 6.

8) Correctness

On all 17 benchmarks, the raw TAC, the SSA form and the optimized form printed identical values; for the kernels these equal the output of the native build and of the Java translation. Every CLIR package was decoded without any table from the encoder, was equal to the optimized program up to a consistent renaming of variables, labels and blocks, and printed the same values when its control-flow graphs were rebuilt and executed. Every Huffman package matched (13) exactly and decompressed to its input. Every columnar container decoded

TABLE 7. Design of CLIR and existing binary program formats

	CLIR	WebAssembly	LLVM bitcode	Java class
Integers	LEB128, byte-aligned	LEB128, byte-aligned	variable-width bit fields	fixed width (1, 2, 4 B)
Operands	SSA registers, tagged literals	operand stack, locals	SSA values, relative ids	operand stack, locals
Literals	pruned pool	immediates	constant blocks	constant pool
Control flow	blocks, ϕ-functions	structured blocks	blocks, ϕ-functions	offsets, stack maps
Metadata	none	types, imports, exports	types, symbols, metadata	names, descriptors, attributes
Entropy coding	optional canonical Huffman	none (transport compression)	abbreviations	none (DEFLATE in JAR)

TABLE 8. Size of the kernels in existing formats (bytes): Java class files (`javac -g:none`) and LLVM bitcode (`clang -Oz`), each also after raw DEFLATE, and their ratio to CLIR. The last row is an empty program.

ID	CLIR	Java class	Java +DEFLATE	Bitcode	Bitcode +DEFLATE	Java/ CLIR	Bitcode/ CLIR
K1	141	689	465	5216	3813	4.9	37.0
K2	174	725	488	5072	3807	4.2	29.1
K3	195	725	492	5824	3889	3.7	29.9
K4	190	749	496	5120	3822	3.9	26.9
K5	189	680	476	5088	3830	3.6	26.9
K6	197	679	473	4960	3696	3.4	25.2
K7	170	727	486	5840	4279	4.3	34.4
K8	106	634	435	4880	3334	6.0	46.0
K9	243	778	523	5296	3998	3.2	21.8
K10	212	761	523	5264	3559	3.6	24.8
Empty	13	311	–	2672	–	–	–

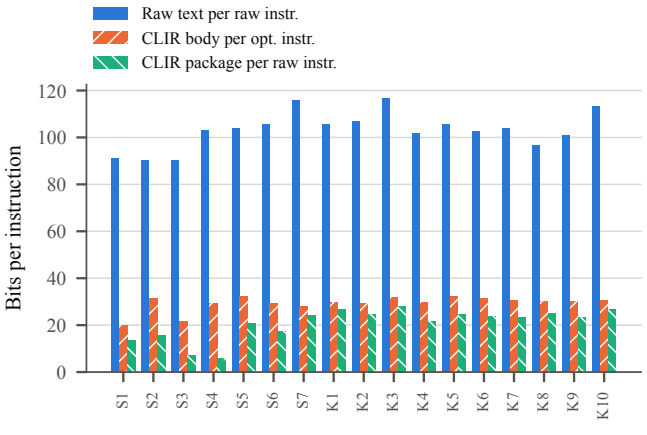


FIGURE 11. Encoding density in bits per instruction, from (16).

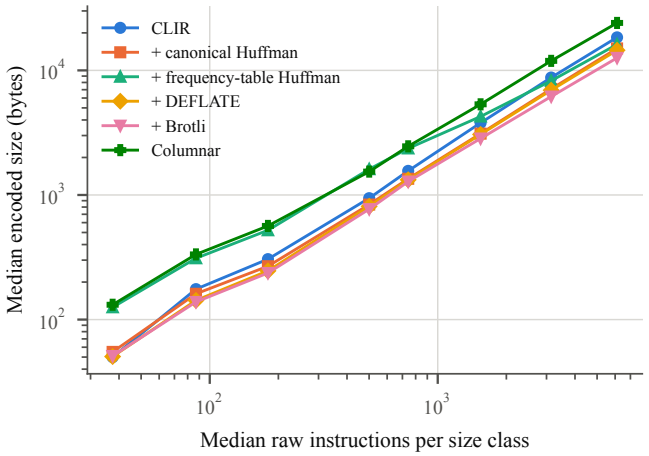


FIGURE 12. Median encoded size against median program size for eight size classes of random programs that read input (eight programs each), for CLIR alone, followed by each coder, and for the columnar container.

TABLE 9. Median running time per stage in milliseconds (30 runs)

ID	Front end	TAC	CFG, dom.	SSA	Opt.	CLIR, Huff.	Colum-nar	Total
S1	0.15	0.07	0.01	0.06	0.21	0.08	0.09	1.12
S2	0.13	0.06	0.03	0.07	0.42	0.08	0.12	1.37
S3	0.16	0.07	0.02	0.06	0.23	0.06	0.08	1.15
S4	0.13	0.05	0.02	0.06	0.22	0.05	0.08	1.01
S5	0.15	0.06	0.02	0.06	0.42	0.09	0.13	1.30
S6	0.14	0.05	0.03	0.05	0.34	0.07	0.11	1.16
S7	0.16	0.05	0.03	0.05	0.35	0.09	0.12	1.27
K1	0.23	0.09	0.05	0.10	0.72	0.15	0.19	2.20
K2	0.28	0.12	0.06	0.13	1.06	0.17	0.22	2.93
K3	0.30	0.12	0.06	0.13	0.99	0.17	0.23	2.90
K4	0.32	0.14	0.06	0.16	1.18	0.18	0.25	3.37
K5	0.28	0.12	0.06	0.14	1.11	0.17	0.24	3.12
K6	0.29	0.13	0.07	0.15	1.23	0.18	0.26	3.33
K7	0.27	0.12	0.05	0.13	0.99	0.16	0.21	2.87
K8	0.20	0.08	0.03	0.08	0.58	0.13	0.16	1.79
K9	0.33	0.17	0.06	0.19	1.43	0.23	0.26	3.96
K10	0.36	0.14	0.07	0.14	1.08	0.20	0.26	3.28

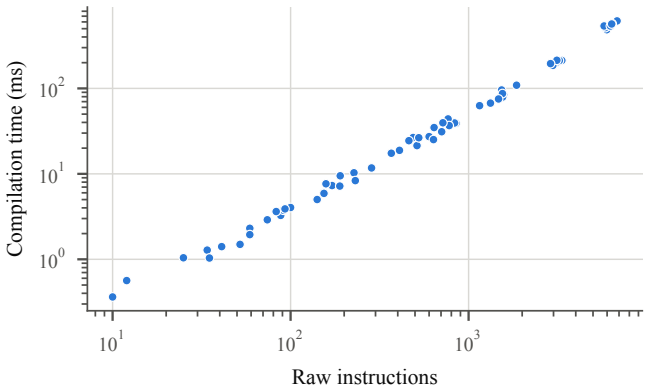


FIGURE 13. End-to-end compilation time of the 64 random programs (minimum of three runs; both axes logarithmic).

back to the optimized instructions. In the first release, the container stored 0 for the 7 of the 14 ϕ-arguments of S2, S5, S6 and S7 that are defined later in the linear order, and its round-trip check compared the encoder's own intermediate records, which a lossy encoding reproduces; signed distances remove the loss, and the check now compares instructions.

Table 10 summarizes differential testing on 1,000 random programs with 12 to 935 raw instructions each, 325,719 in total. The SSA form agreed with the raw TAC on all of them. With the folder of the first release, the optimized form disagreed on 213 programs (21.3%), an agreement rate of only

TABLE 10. Differential testing on random programs: programs whose optimized form printed the same values as the raw TAC. A division divergence disappears when both programs use the floor rule for `/` and `%`.

Optimizer	Input	Programs	Agree	Differ: division	Differ: other
As released: five passes, floor / and %	no	1,000	787	213	0
Five passes, truncating / and %	no	1,000	1,000	0	0
All seven passes	no	1,000	1,000	0	0
All seven passes	yes	500	500	0	0

78.7%. The folder evaluated integer / and % with Python's floor rule, whereas C truncates toward zero, so an expression whose value is -31 reduced modulo 1000 folded to 969 instead of -31; every one of these divergences disappears when both programs are run with the floor rule. The rate is high because our generator often applies % to negative values, but it shows that the defect was not a corner case. The folder now follows (7). With it, the five passes and all seven passes agree with the raw TAC on all 1,000 programs, and all seven passes on 500 programs that read input. All results in this article were produced with the corrected folder, which is part of the released repository. While preparing the kernels we found a second defect in the front end: a temporary could have the name of a program variable of the form `t1` and overwrite it. Benchmark S5 declares such variables but was not miscompiled; temporaries are now named so that the clash cannot occur.

9) Threats to validity

The demonstration programs are small and were written to show the passes, and they have no input; the kernels read input and call functions, but they are short, written for this study and limited to a subset of C without arrays or pointers, so the size of real programs remains to be measured. Integers are unbounded in the pipeline and in the reference interpreters, while C integers overflow; the kernels stay within 32 bits, which their native execution confirms. The shared dictionary was trained on random programs that resemble the demonstration programs, which favors the compressors on those programs. The comparison with Java class files and LLVM bitcode measures formats that carry more information than CLIR. The reference interpreters were written for this study and could share a misconception with the pipeline; native execution of the kernels and their Java translations guard against this for the kernels. All times were measured on one machine.

V. CONCLUSION AND SCOPE FOR FUTURE WORK

This article presented an SSA-based pipeline that compacts three-address code by optimizing it first and encoding it second, and checked every reported number with released evaluation scripts.

Existing methodologies either shrink native code inside large production compilers or compress intermediate code as an opaque file with general-purpose compressors.

The proposed method removes instructions with seven SSA passes, including SCCP and copy propagation, and writes the rest as a self-describing bytecode with one opcode byte, tagged LEB128 operands and a pruned constant pool. Its advantage is that each stage is measured separately, the encoding is compared with compressors applied to the same optimized program, and behavior is verified.

The system was implemented in Python with a Cytoscape.js web front end, and evaluated with independent decoders, reference interpreters, native execution of C kernels and differential testing.

Optimization removed 48.1% to 89.3% of the instructions of seven demonstration programs without input, but only 19.0% to 34.3% of ten kernels that read input and call functions. Against the best of Brotli, Zstandard and xz applied to the optimized text, the CLIR bytecode was 3.4% to 59.7% smaller on the demonstration programs and 39.5% to 50.9% smaller on the kernels; without container headers it lost only on S1, and with a shared dictionary the compressors won on 5 of the 7 small demonstration programs, while CLIR stayed 25.5% to 37.9% smaller on the kernels. Java class files and LLVM bitcode of the kernels were 3.2 to 6.0 and 22 to 46 times larger, mostly because of metadata and fixed per-file costs. With canonical code lengths instead of a frequency table, Huffman coding paid off above about 115 bytes instead of 5.7 kB. Differential testing showed that the folder of the first release miscompiled 213 of 1,000 random programs; the corrected pipeline agreed on all of them and on 500 programs with input.

Future work will extend the language with arrays and memory, so that kernels from an established C benchmark suite can be translated, and will measure the pipeline on them. The entropy stage will use a pre-trained or context-modeling coder, and the bytecode will be compared with WebAssembly binaries produced by a WebAssembly toolchain.

REFERENCES

[1] J. Bauwens, P. Ruckebusch, S. Giannoulis, I. Moerman, and E. De Poorter, "Over-the-air software updates in the Internet of Things: An overview of key principles," *IEEE Commun. Mag.*, vol. 58, no. 2, pp. 35–41, Feb. 2020, doi: 10.1109/MCOM.001.1900125.

[2] World Wide Web Consortium, "WebAssembly core specification," W3C Recommendation, Dec. 2019. [Online]. Available: https://www.w3.org/TR/wasm-core-1/

[3] A. Jangda, B. Powers, E. D. Berger, and A. Guha, "Not so fast: Analyzing the performance of WebAssembly vs. native code," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2019, pp. 107–120.

[4] C. Lattner *et al.*, "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, 2021, pp. 2–14, doi: 10.1109/CGO51591.2021.9370308.

[5] F. Rastello and F. Bouchez Tichadou, Eds., *SSA-Based Compiler Design*. Cham, Switzerland: Springer, 2022, doi: 10.1007/978-3-030-80515-9.

[6] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[7] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," *Proc. IEEE*, vol. 106, no. 11, pp. 1879–1901, Nov. 2018, doi: 10.1109/JPROC.2018.2817118.

[8] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," *ACM Comput. Surv.*, vol. 51, no. 5, pp. 1–42, Sep. 2018, doi: 10.1145/3197978.

[9] A. Moffat, "Huffman coding," *ACM Comput. Surv.*, vol. 52, no. 4, pp. 1–35, Aug. 2019, doi: 10.1145/3342555.
[10] R. C. O. Rocha, P. Petoumenos, Z. Wang, M. Cole, and H. Leather, "Effective function merging in the SSA form," in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2020, pp. 854–868, doi: 10.1145/3385412.3386030.
[11] R. C. O. Rocha, P. Petoumenos, Z. Wang, M. Cole, K. Hazelwood, and H. Leather, "HyFM: Function merging for free," in *Proc. 22nd ACM SIGPLAN/SIGBED Int. Conf. Lang., Compilers, Tools Embedded Syst. (LCTES)*, 2021, pp. 110–121, doi: 10.1145/3461648.3463852.
[12] R. C. O. Rocha, P. Petoumenos, B. Franke, P. Bhatotia, and M. O'Boyle, "Loop rolling for code size reduction," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, 2022, pp. 217–229, doi: 10.1109/CGO53902.2022.9741256.
[13] T. Damásio, V. Pacheco, F. Goes, F. Pereira, and R. Rocha, "Inlining for code size reduction," in *Proc. 25th Brazilian Symp. Program. Lang. (SBLP)*, 2021, pp. 17–24, doi: 10.1145/3475061.3475081.
[14] T. Theodoridis, T. Grosser, and Z. Su, "Understanding and exploiting optimal function inlining," in *Proc. 27th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2022, pp. 977–989, doi: 10.1145/3503222.3507744.
[15] M. Chabbi, J. Lin, and R. Barik, "An experience with code-size optimization for production iOS mobile applications," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, 2021, pp. 363–377, doi: 10.1109/CGO51591.2021.9370306.
[16] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: A machine learning guided compiler optimizations framework," 2021, *arXiv:2101.04808*.
[17] S. Jain, Y. Andaluri, S. VenkataKeerthy, and R. Upadrasta, "POSET-RL: Phase ordering for optimizing size and execution time using reinforcement learning," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2022, pp. 121–131, doi: 10.1109/ISPASS55109.2022.00012.
[18] C. Cummins *et al.*, "Large language models for compiler optimization," 2023, *arXiv:2309.07062*.
[19] C. Cummins *et al.*, "CompilerGym: Robust, performant compiler optimization environments for AI research," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, 2022, pp. 92–105, doi: 10.1109/CGO53902.2022.9741258.
[20] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. P. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 139, 2021, pp. 2244–2253.
[21] J. Chen *et al.*, "A survey of compiler testing," *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–36, Feb. 2020, doi: 10.1145/3363562.
[22] M. N. Wegman and F. K. Zadeck, "Constant propagation with conditional branches," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 2, pp. 181–210, Apr. 1991, doi: 10.1145/103135.103136.
[23] E. S. Schwartz and B. Kallick, "Generating a canonical prefix encoding," *Commun. ACM*, vol. 7, no. 3, pp. 166–169, Mar. 1964, doi: 10.1145/363958.363991.
[24] A. Hilbig, D. Lehmann, and M. Pradel, "An empirical study of real-world WebAssembly binaries: Security, languages, use cases," in *Proc. Web Conf. (WWW)*, 2021, pp. 2696–2708, doi: 10.1145/3442381.3450138.
[25] J. Cabrera Arteaga *et al.*, "Superoptimization of WebAssembly bytecode," in *Companion Proc. 4th Int. Conf. Art, Sci., Eng. Program. (⟨Programming⟩)*, 2020, pp. 36–40, doi: 10.1145/3397537.3397567.
[26] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, "Alive2: Bounded translation validation for LLVM," in *Proc. 42nd ACM SIGPLAN Int. Conf. Program. Lang. Des. Implement. (PLDI)*, 2021, pp. 65–79, doi: 10.1145/3453483.3454030.
[27] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, pp. 1–25, Nov. 2020, doi: 10.1145/3428264.
[28] J. Ernst, W. Evans, C. W. Fraser, T. A. Proebsting, and S. Lucco, "Code compression," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 1997, pp. 358–365, doi: 10.1145/258915.258947.
[29] M. Franz and T. Kistler, "Slim binaries," *Commun. ACM*, vol. 40, no. 12, pp. 87–94, Dec. 1997, doi: 10.1145/265563.265576.
[30] W. Pugh, "Compressing Java class files," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 1999, pp. 247–258, doi: 10.1145/301618.301676.
[31] S. Lucco, "Split-stream dictionary program compression," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2000, pp. 27–34, doi: 10.1145/349299.349307.
[32] W. S. Evans and C. W. Fraser, "Bytecode compression via profiled grammar rewriting," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2001, pp. 148–155, doi: 10.1145/378795.378827.
[33] S. K. Debray, W. Evans, R. Muth, and B. De Sutter, "Compiler techniques for code compaction," *ACM Trans. Program. Lang. Syst.*, vol. 22, no. 2, pp. 378–415, Mar. 2000, doi: 10.1145/349214.349233.
[34] A. Haas *et al.*, "Bringing the web up to speed with WebAssembly," in *Proc. 38th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2017, pp. 185–200, doi: 10.1145/3062341.3062363.
[35] LLVM Project, "LLVM bitcode file format," 2024. [Online]. Available: https://llvm.org/docs/BitCodeFormat.html
[36] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and D. Smith, *The Java Virtual Machine Specification, Java SE 21 Edition*. Redwood City, CA, USA: Oracle, 2023. [Online]. Available: https://docs.oracle.com/javase/specs/jvms/se21/html/
[37] Python Software Foundation, "Python 3.13 documentation," 2024. [Online]. Available: https://docs.python.org/3.13/
[38] M. Franz *et al.*, "Cytoscape.js 2023 update: A graph theory library for visualization and analysis," *Bioinformatics*, vol. 39, no. 1, Jan. 2023, Art. no. btad031, doi: 10.1093/bioinformatics/btad031.
[39] J. Alakuijala *et al.*, "Brotli: A general-purpose data compressor," *ACM Trans. Inf. Syst.*, vol. 37, no. 1, pp. 1–30, Jan. 2019, doi: 10.1145/3231935.
[40] Y. Collet and M. Kucherawy, "Zstandard compression and the 'application/zstd' media type," Internet Eng. Task Force (IETF), RFC 8878, Feb. 2021, doi: 10.17487/RFC8878.
[41] Tukaani Project, "XZ Utils," 2024. [Online]. Available: https://github.com/tukaani-project/xz

AUTHOR CONTRIBUTIONS

Hrituraj Deb: methodology, software (revision), validation, formal analysis, writing (original draft, review and editing). **Rishabh Kumar:** software (compiler core, server and web front end), visualization, writing (first draft). **Gautham G:** validation (test programs, benchmark checks), visualization (diagrams, screenshot). **Aarnav Bhasin:** validation (test programs, benchmark checks), visualization (diagrams, screenshot). All authors read and approved the final manuscript.

ACKNOWLEDGEMENTS

The compiler evaluated in this article was designed and implemented by the authors. The authors used the AI assistant Claude (Anthropic) to draft the manuscript text and to write the evaluation scripts and the figures of Section IV. In the revision, the authors also used it, under their direction, to implement the changes requested by the reviewers: the corrected constant folder, SCCP and copy propagation, the self-describing bytecode with a pruned constant pool, canonical Huffman coding, the corrected columnar container, function and input support, the kernels and the extended evaluation. All such material was checked against the measured results and revised by the authors, who take full responsibility for the content of this article.

FUNDING

This work received no external funding.

COMPETING INTERESTS

The authors declare no competing interests.

DATA AND CODE AVAILABILITY

The source code of the compiler, the benchmarks and the evaluation scripts that produce every table, figure and number in this article are available at https://github.com/rishbCLN/compiler_project, tag `paper-revision-1`. The scripts are in the directory `paper_eval`, together with the measured results; `python3 paper_eval/run_all.py` repeats the evaluation, and `paper_eval/README.md` describes each script and the software versions used.

HRITURAJ DEB is a third-year student of the B.Tech. degree program in Computer Science and Engineering (Core) at the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

GAUTHAM G is a third-year student of the B.Tech. degree program in Computer Science and Engineering (Core) at the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

RISHABH KUMAR is a third-year student of the B.Tech. degree program in Computer Science and Engineering (Core) at the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

AARNAV BHASIN is a third-year student of the B.Tech. degree program in Computer Science and Engineering with a specialization in Data Science at the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.